

Codeforces Premium

Product Requirements Document (PRD)

Vision

Create the most premium and intuitive competitive programming experience available for Codeforces.

The extension should not feel like a browser extension or a collection of injected CSS. Instead, it should feel like a professional desktop application built specifically for competitive programmers while preserving the complete functionality of Codeforces.

The primary objective is to eliminate friction during problem solving. Every interaction should feel fast, polished, predictable, and enjoyable.

The extension must never compromise the integrity of Codeforces. It enhances the user interface and workflow while respecting the existing platform.

Core Principles

- Premium UI with zero clutter
 - No unnecessary page redirects
 - Lightning-fast interactions
 - Native desktop application feel
 - Keyboard-first workflow
 - Beautiful typography
 - Highly customizable
 - Privacy-first
 - All personal data stored locally
 - Smooth animations without sacrificing performance
 - Consistent design across every page
-

Local-First Architecture

The extension stores all user preferences locally.

No user account is required.

No cloud synchronization is required.

No external database is required.

Everything remains on the user's computer, including:

- Notes
- Bookmarks
- Theme
- Editor preferences
- Layout preferences
- Custom test cases
- Practice history cache
- Recent problems
- Keyboard preferences
- Window layout
- Font settings
- Console history

Complete UI Redesign

Every Codeforces page should be transformed into a modern interface.

The redesign includes:

- Modern navigation
- Glassmorphism and subtle shadows
- Rounded components
- Smooth transitions
- Professional spacing
- Better typography
- Responsive layouts
- Consistent color palette
- Premium icons
- Better information hierarchy

The user should immediately feel like they are using a premium application rather than a traditional website.

Adaptive Themes

Support multiple themes.

Examples:

- Light
- Dark
- OLED Black
- GitHub
- Dracula
- Nord
- Catppuccin
- Solarized
- Custom Accent Color

Allow users to customize:

- Accent color
- Border radius
- Transparency
- Blur strength
- Shadows
- Animation speed

Premium Monaco Editor

Replace the default editor with a fully customized Monaco Editor.

Features include:

- Syntax highlighting
- Line numbers
- Current line highlight
- Sticky cursor
- Auto indentation
- Auto formatting
- Rainbow brackets
- Bracket matching
- Auto closing brackets
- Auto closing quotes
- Code folding
- Sticky scroll
- Word wrapping
- Font ligatures
- Smooth cursor animation
- Minimap
- Breadcrumbs
- Multi cursor

- Rename symbol
- Find and replace
- Format document
- Comment shortcuts
- Auto save draft
- Persistent undo history
- Adjustable font size
- Adjustable tab size

Support Vim mode.

Support Emacs mode.

Support multiple programming languages exactly as Codeforces supports.

Smart Layout

Resizable split-screen layout.

Problem statement on one side.

Editor on the other.

Users can drag the divider.

The layout automatically remembers previous sizes.

Users can switch between:

- Vertical split
 - Horizontal split
 - Editor only
 - Problem only
-

Better Problem Statement Rendering

Improve readability dramatically.

Features include:

- Beautiful headings
- Better paragraph spacing

- Better code block rendering
- Responsive tables
- Responsive images
- Automatic word wrapping
- Inline mathematical expressions
- Better list formatting
- Better constraints section
- Better input/output formatting
- Sticky section navigation

Long lines should wrap naturally instead of overflowing.

No text should ever be clipped.

Problem Navigation Sidebar

Automatically generate a navigation panel.

Examples:

- Description
- Input
- Output
- Constraints
- Examples
- Notes

Clicking any section scrolls instantly.

The current section remains highlighted.

Smart Example Viewer

Each sample test becomes an interactive component.

Users can:

- Switch between examples
- Copy input
- Copy output
- Copy expected answer
- Run selected example
- Edit sample

- Duplicate sample
 - Save as custom test
-

Advanced Custom Test Manager

Instead of redirecting to another page, users can manage custom tests directly inside the extension.

Features include:

- Multiple custom tests
 - Rename tests
 - Duplicate tests
 - Delete tests
 - Import tests
 - Export tests
 - Persistent local storage
 - Batch execution
 - Input validation
-

In-Place Run System

Running code should never redirect the user away from the workspace.

Instead:

- Open integrated execution panel
 - Display console
 - Show runtime
 - Show output
 - Show expected output
 - Highlight differences
 - Display compile errors clearly
 - Keep editor focused
-

In-Place Submission Experience

Submitting code should remain inside the premium interface.

The workflow includes:

- Animated submission state
- Queue status
- Live judging updates
- Final verdict
- Runtime
- Memory
- Submission ID
- Direct submission details panel

No unnecessary navigation.

Premium Verdict UI

Each verdict has a unique appearance.

Accepted

- Green animation
- Success icon

Wrong Answer

- Red animation
- Difference viewer

Compilation Error

- Yellow warning
- Compiler output formatting

Runtime Error

- Stack trace formatting

Time Limit

- Performance hints

Memory Limit

- Memory visualization

Presentation Error

- Whitespace comparison

Submission History

Display submissions as modern cards.

Each submission displays:

- Verdict
- Runtime
- Memory
- Language
- Submission time
- Contest
- Problem
- Tags

Support searching and filtering.

Smart Diff Viewer

For Wrong Answer cases.

Display:

Your Output

Expected Output

Differences highlighted.

Whitespace differences highlighted.

Character-level comparison.

Line-level comparison.

Contest Dashboard

Redesign contest pages.

Display:

- Problem ratings
 - Solved count
 - Success rate
 - Estimated difficulty
 - Tags
 - Personal status
 - Contest timer
 - Rank
 - Penalties
-

Practice Dashboard

Create a personal dashboard.

Track:

- Problems solved
 - Average difficulty
 - Rating progression
 - Daily streak
 - Weekly streak
 - Monthly streak
 - Topic mastery
 - Contest participation
 - Languages used
-

Notes System

Every problem supports notes.

Users can:

- Write markdown
- Pin notes
- Search notes
- Export notes
- Organize notes

Everything remains local.

Bookmark System

Bookmark problems.

Create folders.

Add custom labels.

Search bookmarks.

Filter bookmarks.

Smart Search

Search instantly by:

- Problem name
 - Tags
 - Contest
 - Difficulty
 - Notes
 - Bookmarks
-

Keyboard Shortcuts

Provide a complete keyboard-first experience.

Examples include:

- Submit
- Run
- Format
- Search
- Open command palette
- Toggle sidebar
- Toggle terminal
- Switch examples
- Open notes
- Increase font
- Decrease font

All shortcuts should be customizable.

Command Palette

A universal command interface similar to VS Code.

Users can quickly:

- Open settings
- Search problems
- Toggle theme
- Switch language
- Run code
- Submit code
- Open notes
- Open bookmarks

Settings Center

One centralized settings page.

Categories include:

General

Editor

Appearance

Fonts

Animations

Shortcuts

Workspace

Privacy

Accessibility

Experimental

Accessibility

Support:

- Screen readers
 - High contrast mode
 - Large fonts
 - Keyboard navigation
 - Reduced motion
 - Dyslexia-friendly fonts
-

Workspace Persistence

When reopening a problem, restore:

- Cursor position
 - Scroll position
 - Split sizes
 - Notes
 - Selected language
 - Open console
 - Editor state
 - Undo history
-

Performance

The extension should feel instantaneous.

Requirements:

- Smooth scrolling
 - No layout shifts
 - Fast startup
 - Lazy loading
 - Minimal memory usage
 - Efficient DOM updates
 - Hardware-accelerated animations
-

Privacy

The extension should clearly communicate:

- No tracking
 - No analytics
 - No advertisements
 - No user accounts
 - No unnecessary permissions
 - Local-first storage
-

Optional AI Assistant

An optional AI companion that activates only when the user explicitly requests it.

Capabilities include:

- Explain problem statements
- Explain constraints
- Analyze accepted solutions
- Analyze failed submissions
- Suggest optimizations
- Explain algorithm complexity
- Recommend similar problems
- Generate study plans
- Detect common mistakes
- Provide learning resources

The AI should never automatically reveal complete contest solutions or compromise the fairness of ongoing contests.

Visual Polish

Every interaction should include subtle polish.

Examples:

- Smooth hover effects
- Premium loading skeletons
- Beautiful transitions
- Micro-interactions
- Elegant empty states

- Refined icons
 - Consistent spacing
 - Smooth panel resizing
 - Animated notifications
 - Soft shadows
 - Crisp typography
 - Native-feeling dialogs
-

Product Goal

The final experience should make users forget they are interacting with the traditional Codeforces interface. Every page should feel like a thoughtfully designed, high-performance competitive programming environment where solving problems is faster, more comfortable, and more enjoyable than on any existing Codeforces enhancement.

The benchmark is not to imitate another platform, but to become the definitive premium client for Codeforces.